

Customer Churn Analysis and Prediction

Task 1

Data Preparation

Internship Report

Dataset: Telco Customer Churn Dataset

Table of Contents

1. Introduction	3
2. Dataset Overview	3
2.1 Feature Dictionary	3
3. Initial Data Inspection	4
4. Data Cleaning	4
4.1 Fixing the TotalCharges Data Type	4
4.2 Handling Missing Values	4
4.3 Removing Irrelevant Columns	5
5. Categorical Variable Encoding.....	5
5.1 Binary Columns.....	5
5.2 Multi-Class Columns (One-Hot Encoding).....	5
6. Splitting the Dataset	5
7. Conclusion.....	5

1. Introduction

This report covers Task 1 of the Customer Churn Analysis and Prediction project: Data Preparation. The goal of this task was to load the raw dataset, explore its structure, handle missing values, convert categorical variables into a numerical representation, and split the data into training and testing sets so that it is ready for the modeling work carried out in later tasks. The sections below describe the dataset, the steps I followed, the decisions I made at each step, and why.

2. Dataset Overview

The project uses the Telco Customer Churn Dataset, a widely used benchmark dataset for churn prediction. After loading the dataset with pandas, an initial inspection showed that it contains 7,043 customer records described by 21 columns, covering demographic attributes, the services each customer has subscribed to, account/billing information, and the target variable that indicates whether the customer churned.

2.1 Feature Dictionary

Before doing any cleaning or transformation, I went through every column to understand what it represents. This step is important because it determines how each variable should later be encoded and interpreted during the analysis. The table below summarizes the variables in the dataset.

Feature	Possible Values	Description
customerID	Unique ID	Unique identifier for each customer; dropped before modeling.
gender	Female / Male	Customer's gender.
SeniorCitizen	0 / 1	Whether the customer is a senior citizen.
Partner	Yes / No	Whether the customer has a partner or spouse.
Dependents	Yes / No	Whether the customer has dependents (e.g. children, family).
tenure	Number of months	How many months the customer has stayed with the company.
PhoneService	Yes / No	Whether the customer subscribes to phone service.
MultipleLines	Yes / No / No phone service	Whether the customer has more than one phone line.
InternetService	DSL / Fiber optic / No	Type of internet service the customer uses.
OnlineSecurity	Yes / No / No internet service	Whether the customer has online security add-on.
OnlineBackup	Yes / No / No internet service	Whether the customer has online data backup.
DeviceProtection	Yes / No / No internet service	Whether the customer has device protection.
TechSupport	Yes / No / No internet service	Whether the customer has technical support.

Feature	Possible Values	Description
StreamingTV	Yes / No / No internet service	Whether the customer streams TV through the service.
StreamingMovies	Yes / No / No internet service	Whether the customer streams movies through the service.
Contract	Month-to-month / One year / Two year	The customer's contract type.
PaperlessBilling	Yes / No	Whether the customer uses paperless billing.
PaymentMethod	Electronic check / Bank transfer / Credit card / Mailed check	How the customer pays.
MonthlyCharges	Numeric	Amount billed to the customer each month.
TotalCharges	Numeric	Total amount charged to the customer since joining.
Churn (target)	Yes / No	Whether the customer left the company.

3. Initial Data Inspection

Using `df.info()`, I confirmed that the dataset contains 7,043 rows and 21 columns, and that most columns (18 of them) are categorical, while only a few are numerical. No column showed an explicit count of null values at this stage. However, this first inspection also revealed two issues that needed to be addressed before the data could be used for modeling:

- TotalCharges was stored as an object (text) column instead of a numeric type, even though it represents a monetary amount.
- Churn, the target variable, is categorical (Yes/No) and needs to be encoded numerically for machine learning.

4. Data Cleaning

4.1 Fixing the TotalCharges Data Type

I converted the TotalCharges column using `pd.to_numeric(df['TotalCharges'], errors='coerce')`, which forces the column to a numeric type and turns any value that cannot be converted into a missing value (NaN) instead of raising an error. Re-running `df.info()` afterward confirmed that the column was now correctly typed as float.

4.2 Handling Missing Values

I then checked for missing values across the whole dataset using `df.isnull().sum()`. Every column returned zero missing values except for TotalCharges, which had 11 missing entries — exactly the records that could not be converted to numeric in the previous step. Since this represents a very small fraction of the 7,043 records, I filled these missing values with 0 using `df['TotalCharges'].fillna(0)` rather than dropping the rows.

4.3 Removing Irrelevant Columns

Before encoding, I created a working copy of the cleaned dataframe and removed the customerID column, since it is a unique identifier that carries no predictive information and would only add noise to the model.

5. Categorical Variable Encoding

To prepare the categorical columns for machine learning, I first inspected the unique values of every object-type column. This showed that the categorical variables naturally fall into two groups, which I treated differently.

5.1 Binary Columns

Six columns only take two values (e.g. Yes/No or Female/Male): gender, Partner, Dependents, PhoneService, PaperlessBilling, and Churn. These were mapped directly to 0 and 1.

5.2 Multi-Class Columns (One-Hot Encoding)

Ten columns have more than two categories: MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, and PaymentMethod. For these, I applied One-Hot Encoding using `pd.get_dummies(churn_df_copy, drop_first=True)`, which creates a separate binary column for each category while dropping one category per feature to avoid redundancy (the dummy variable trap).

Why drop_first=True?

Dropping the first category per feature avoids perfect multicollinearity between the dummy columns, which keeps the encoded feature set suitable for models such as Logistic Regression.

6. Splitting the Dataset

Once encoding was complete, I separated the features (X) from the target variable (y = Churn_Yes) and used `train_test_split` from scikit-learn to divide the data into an 80% training set and a 20% testing set, with `random_state=42` to make the split reproducible. The training set is used to fit the models in Task 4, while the testing set is kept aside to evaluate how well those models generalize to unseen customers.

7. Conclusion

By the end of this task, the dataset was fully understood, cleaned, and transformed: the TotalCharges type issue and its 11 missing values were resolved, the non-informative customerID column was removed, all categorical variables were encoded numerically, and the data was split into training and testing subsets. This clean, encoded dataset forms the foundation for the exploratory analysis, segmentation, and predictive modeling carried out in the following tasks.